

RailConnect 2000

User Manual and Technical Reference

Network Rail Delay Propagation System

Application scope

Timetable and TRUST movement ingestion; train-level delay analysis; inferred propagation-network construction; validation; constrained recovery prioritisation; and report export.

Project author: Christopher Blaxland-Kay

Manual revision 1.2

27 September 2026

Development prototype – source-mode Windows application

How to use this manual

This document serves two audiences simultaneously. A first-time operator can follow the numbered instructions without needing to know Python, SQL, graph theory, statistics, or optimisation. A technical reader can use the same chapters as a precise specification of what the application calculates.

Each functional chapter therefore follows the same pattern:

1. **Purpose:** what the page is for.
2. **Prerequisites:** what must already exist.
3. **Controls:** what every field and button does.
4. **Procedure:** the exact sequence to follow.
5. **Completion:** how to know the operation has finished.
6. **Technical method:** the implemented mathematics or data logic.
7. **Interpretation:** what may and may not be concluded.
8. **Outputs and errors:** where results are stored and how to recover.

Interpretation warning

The words *causal*, *propagation*, *benefit*, *cost*, and *recovery* refer to the application’s modelling constructs. They do not mean that operational causation or intervention performance has been verified against Network Rail control-room records. The application is a research and decision-support prototype, not a safety-critical railway control system.

Important

Compatibility statement. This revision describes the development source snapshot audited on 25 September 2026. That snapshot was not associated with a numbered software release tag. Before publishing a later copy, replace this statement with the exact release number or Git commit and recheck every control, table name, default, and formula against that version.

Independent-project statement. RailConnect 2000 is an independent research and portfolio project. It is not affiliated with, endorsed by, or an official product of Network Rail. Network Rail names and data-source terms are used only to identify the origin and technical meaning of the public feeds.

Reading convention. Button labels appear in **bold**; field labels appear in *italics*; commands, paths, identifiers, and table names appear in `monospacedtext`. Values shown in interface illustrations are dataset-specific examples, not expected results.

Contents

How to use this manual	i
1 Purpose, scope, and analytical philosophy	1
1.1 What the application does	1
1.2 What the application does not do	1
1.3 The end-to-end dependency chain	2
2 Terminology and notation	3
2.1 Railway and data terms	3
2.2 Mathematical notation	4
3 Installation and starting the application	5
3.1 Prerequisites	5
3.2 One-time environment setup	5
3.3 Starting the application on a later day	6
3.4 Safe shutdown	6
4 Workspace, files, and persistence	7
4.1 Choosing the data directory	7
4.2 Resulting directory structure	7
4.3 What persists between sessions	7
5 Application shell and common behaviour	9
5.1 Navigation and status	9
5.2 Display modes	9
5.3 How completion is indicated	9
6 Connection page	10
6.1 Purpose and controls	10
6.2 Testing credentials	10
6.3 Credential handling	11

7 Live Feed page	12
7.1 Purpose	12
7.2 Starting and stopping capture	12
7.3 Technical connection details	12
8 Database page	14
8.1 Purpose and safe order of operations	14
8.2 Downloading the full timetable	14
8.3 Building the timetable database	14
8.4 Loading a movement file	15
8.5 Core database schema	15
8.5.1 Timetable tables	15
8.5.2 Movement table field groups	16
9 Train Analysis page	17
9.1 Purpose and prerequisites	17
9.2 Selecting a train	17
9.3 Event reconstruction	17
9.4 Metric cards and classification	18
9.5 Chart and event table	18
10 Propagation Analysis page	19
10.1 Purpose	19
10.2 Parameters	19
10.3 Step 1: location enrichment	19
10.4 Step 2: constructing temporally ordered pairs	20
10.5 Step 3: affected-train sequence context	20
10.6 Step 4: candidate and score construction	20
10.7 Step 5: directed edge aggregation	21
10.8 Step 6: node and location summaries	21
10.9 Metrics and result tabs	21
10.10 Network explorer	21
10.11 Completion and interpretation	22
11 Validation Laboratory	24
11.1 Purpose and prerequisites	24
11.2 Controls	24
11.3 Matched-control method	24

11.3.1	Sequence preparation	24
11.3.2	Reconstructing a propagation case	25
11.3.3	Building the local control pool	25
11.3.4	Classification rules	26
11.4	Threshold-sensitivity grid	26
11.5	Negative and stress tests	26
11.6	Reading the page	27
11.7	Validation outputs	27
12	Recovery Optimisation	28
12.1	Purpose and warning	28
12.2	Controls	28
12.3	Train-level ranking	29
12.4	Location-level ranking	29
12.5	Candidate cost and binary programme	29
12.6	Exact dynamic-programming solver	29
12.7	Reading the page and outputs	30
13	Reports and Exports	31
13.1	Inventory	31
13.2	Generating an HTML report	31
13.3	Exporting to Downloads	31
13.4	Report security and interpretation	32
14	Complete End-to-End Workflow	33
14.1	Phase 1: start and establish the workspace	33
14.2	Phase 2: acquire source data	33
14.3	Phase 3: build the local database	34
14.4	Phase 4: inspect data quality before modelling	34
14.5	Phase 5: construct the propagation network	34
14.6	Phase 6: validate the inferred relationships	35
14.7	Phase 7: explore a constrained recovery plan	35
14.8	Phase 8: generate and preserve the report	35
14.9	End-to-end acceptance checklist	35
15	Empirical Case Study	37
15.1	Purpose and evidence boundary	37
15.2	Recorded configuration	37

15.3 Scale and analytical funnel	38
15.4 Location-level structure	39
15.5 Edge ranking versus validation	39
15.6 Validation and robustness results	39
15.7 Recovery optimisation	41
15.8 What the case study demonstrates	41
15.9 Reproducing this case study more completely	43
16 Data Lifecycle and Reproducibility	44
16.1 What each operation changes	44
16.2 Why rerun order matters	44
16.3 Minimum experiment record	45
17 Database and Output Reference	46
17.1 SQLite table inventory	46
17.2 Movement-event fields	47
17.3 Propagation-pair field groups	47
17.4 Propagation edge, node, and location fields	48
17.5 Validation output fields	48
17.6 Recovery output fields	49
17.7 Filesystem output inventory	49
18 Technical Architecture and Runtime Behaviour	51
18.1 Application layers	51
18.2 Threading and close protection	51
18.3 Core dependencies	51
18.4 Network and endpoint behaviour	52
18.5 Performance characteristics	52
19 Troubleshooting and Recovery Procedures	53
19.1 First diagnostic sequence	53
19.2 Startup and environment problems	53
19.3 Connection and live-feed problems	54
19.4 Timetable and database problems	54
19.5 Analysis-stage problems	55
19.6 Chart, table, report, and export problems	56
19.7 When to stop rather than retry	57

20 Security, Privacy, and Safe Operation	58
20.1 Credential model	58
20.2 Local storage and backups	58
20.3 Network trust boundary	58
20.4 Data-source terms and attribution	59
20.5 Operational safety	59
21 Statistical Interpretation and Model Limitations	60
21.1 Descriptive quantities are not causal estimates	60
21.2 Potential confounding and dependence	60
21.3 Selection and measurement limitations	60
21.4 Raw variation and status limitation	61
21.5 Timetable-integration limitation	61
21.6 Matched-control limitations	61
21.7 Sensitivity and negative tests	62
21.8 Graph limitations	62
21.9 Optimisation limitations	62
21.10 Responsible wording for results	62
22 Default Settings and Quick Reference	63
22.1 Default controls	63
22.2 Fixed internal constants	63
22.3 Startup command card	64
23 Frequently Asked Questions	65
A Glossary	67
B Sources and Version Basis	69
C Formula Summary	70

Purpose, scope, and analytical philosophy

1.1 What the application does

RailConnect 2000 is a local PySide6 desktop application wrapped around a staged railway movement-analysis pipeline. It acquires and stores two public Network Rail data sources, but their current analytical roles are different:

- the full CIF timetable download supplies TIPLOC/STANOX reference data used to give movement locations readable names; the application also stores schedule and schedule-location records for future work;
- live TRUST movement messages received from the `/topic/TRAIN_MVT_ALL_TOC` STOMP topic supply the event sequences, timestamps, timetable-variation values, and status fields used by the current train, propagation, validation, and recovery stages.

The application converts those inputs into an SQLite workspace, reconstructs the observed variation sequence of individual trains, constructs candidate train-to-train propagation relationships, compares those relationships with local controls and alternative thresholds, and selects a limited set of prototype recovery priorities.

Important

The current analytical services do *not* match individual TRUST train IDs to CIF schedules, filter schedules by operating day, reconstruct planned CIF journeys, or use the `schedules` and `schedule_locations` tables in the propagation model. Those tables are loaded and retained, but current downstream use of CIF is limited to location-name enrichment through `tiploc`.

1.2 What the application does not do

The current version does not directly observe or model:

- signalling block occupation or route setting;
- dispatching instructions and control-room decisions;
- exact platform conflicts;
- crew, rolling-stock, or unit diagrams;
- incident attribution codes sufficient to establish ground-truth causality;

- passenger demand, delay minutes, compensation, or monetary cost;
- a microscopic infrastructure simulation;
- a production-grade operational recovery plan.

Consequently, the central analytical claim is deliberately modest:

The application identifies temporally ordered, same-location train pairs for which the stored source and affected variation values exceed selected thresholds and the affected train's stored variation increases more strongly after the encounter than before it.

That is evidence for a *candidate pathway*. It is not, by itself, proof that the earlier train caused the later train's delay.

1.3 The end-to-end dependency chain

The pages are intended to be used from top to bottom:

Page	Depends on and produces
Connection	Supplies session credentials and the selected data directory.
Live Feed	Uses credentials; produces timestamped movement JSONL.
Database	Downloads CIF, builds <code>railway.db</code> , and imports movement JSONL.
Train Analysis	Reads movement events and uses TIPLOC/STANOX reference data for display names.
Propagation	Reads movement events and the timetable-derived location lookup; creates enriched, pair, edge, node, and location tables.
Validation	Reads enriched movements and propagation edges; creates three validation tables.
Recovery	Reads propagation edges; creates ranked plans, a binary solution, and a summary.
Reports	Reads whichever result tables currently exist; produces HTML and CSV exports.

Important

If a later page reports that a table is missing, return to the earliest prerequisite page and run that stage. **Load existing results** reads the most recently stored result tables; it does not calculate new results and does not apply parameters that have merely been changed on screen.

Terminology and notation

2.1 Railway and data terms

Term	Meaning in this application
CIF	Common Interface Format timetable data, downloaded here as gzipped JSON records.
TRUST	Train Running System movement information exposed through Network Rail's open-data feed.
STOMP	Messaging protocol used to subscribe to the live TRUST topic.
STANOX	Numeric/character location identifier carried by movement messages.
TIPLOC	Timetable location code. The CIF location table links timetable descriptions and, where available, STANOX identifiers.
CRS	Three-letter customer-facing station code where present in the timetable record.
Train ID	TRUST identifier used to group movement messages into an observed train sequence.
Movement event	An observed arrival, departure, or other movement message for one train at one location and time.
Raw timetable variation	Numeric number of minutes supplied in <code>timetable_variation</code> . Network Rail supplies direction separately in <code>variation_status</code> .
Variation status	Feed label such as <code>EARLY</code> , <code>LATE</code> , <code>ONTIME</code> , or <code>OFFROUTE</code> .
Analysis variation (V)	In the audited build, the raw numeric <code>timetable_variation</code> used directly in calculations, without sign conversion from <code>variation_status</code> .
Source train	Earlier train in a candidate same-location pair.
Affected train	Later train in that pair.
Interaction	The proposed temporal encounter at the shared location.
Edge	Directed aggregate relation from a source train to an affected train.
Control	Nearby same-location, same-direction train event excluded from the candidate interaction rule and used as a comparator.
Intervention	Prototype choice to prioritise a source train or location.

Known analytical limitation

The feed documents the numeric variation and the early/late status as separate fields. The audited build stores both but uses the numeric value directly in all thresholds, differences, charts, and scores. It does not convert **EARLY** values to negative signed lateness or exclude **OFFROUTE** values. Therefore (V) throughout this manual means the value actually used by the code; it must not be described as verified signed delay.

Before making substantive delay claims, inspect representative raw records and implement a documented normalisation rule. A plausible future rule would map **EARLY** to a negative value, **LATE** to a positive value, **ONTIME** to zero, and handle **OFFROUTE**/missing/inconsistent rows separately. That rule is *not* implemented in the version documented here. After any correction, regenerate every downstream result.

2.2 Mathematical notation

For an affected train's ordered event sequence, write

V^- = stored analysis variation at the event before the interaction,

V^0 = stored analysis variation at the interaction event,

V^+ = stored analysis variation at the next event.

The implemented changes are

$$W_{\text{pre}} = V^0 - V^-, \quad (2.1)$$

$$W_{\text{post}} = V^+ - V^0, \quad (2.2)$$

$$W_{\text{inc}} = W_{\text{post}} - W_{\text{pre}}. \quad (2.3)$$

For any real number z ,

$$[z]_+ = \max(z, 0). \quad (2.4)$$

Indicator notation $\mathbf{1}\{A\}$ equals one when statement A is true and zero otherwise.

Installation and starting the application

3.1 Prerequisites

The current source-mode application requires:

- a Windows computer with Python 3 installed and available as `python`;
- the project folder, downloaded from GitHub or obtained as a ZIP;
- an internet connection for authentication, timetable download, and live capture;
- Network Rail Open Data credentials;
- sufficient disk space for the full timetable, captured messages, and SQLite database.

The repository does not currently declare or pin a supported Python minor version. Do not invent one in a report. Record the version actually used:

```
python --version
```

If the project has not already been downloaded, one option is:

```
git clone https://github.com/faust2/Network-Rail-Delay-Propagation-Pipeline.git
cd Network-Rail-Delay-Propagation-Pipeline
```

Alternatively, download the repository ZIP from GitHub, extract it, and open the extracted project folder rather than an individual Python file.

3.2 One-time environment setup

Open Command Prompt in the project folder and run:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install PySide6 requests stomp.py pandas matplotlib networkx
```

The virtual environment isolates the application's packages from other Python projects. It does not need to be recreated whenever the computer starts. The package versions are not currently pinned, so record installed versions for a reproducible experiment with `python -m pip freeze`.

3.3 Starting the application on a later day

```
cd "C:\path\to\Network-Rail-Delay-Propagation-Pipeline"  
.venv\Scripts\activate  
python -m desktop_app.main
```

Replace the path if the project has been moved. The prompt should show `(.venv)` before the launch command. Keep the terminal open while using the source-mode application; unexpected Python errors are printed there.

3.4 Safe shutdown

Close the main window normally. The application prevents closure while a database, train-analysis, propagation, validation, recovery, or report worker is running. If live capture is active, shutdown requests the feed worker to stop and disconnect.

Workspace, files, and persistence

4.1 Choosing the data directory

The *Data directory* on the Connection page is the folder in which the application expects `railway.db` and creates the `raw` subfolder. For the cleanest layout, select the project's `data` folder. If `data` is selected, reports and output tables are placed in an `outputs` folder beside it.

4.2 Resulting directory structure

```
Network Rail Delay Propagation Project\
|-- data\
|   |-- railway.db
|   |-- raw\
|       |-- schedule_full.json.gz
|       |-- train_movements_YYYYMMDD_HHMMSS.jsonl
|-- outputs\
|   |-- tables\
|       |-- validation_control_comparison.csv
|       |-- validation_threshold_sensitivity.csv
|       |-- validation_negative_tests.csv
|       |-- recovery_intervention_plan_by_train.csv
|       |-- recovery_intervention_plan_by_location.csv
|       |-- recovery_intervention_bip_solution.csv
|       |-- recovery_intervention_bip_summary.csv
|   |-- reports\
|       |-- railconnect-report-YYYYMMDD-HHMMSS\
|           |-- railconnect_report.html
|           |-- tables\
|               |-- [full report CSV extracts]
```

Exporting a report to Downloads additionally creates `railconnect-report-YYYYMMDD-HHMMSS.zip` in the operating system's Downloads folder.

4.3 What persists between sessions

The following persist because they are written to disk:

- downloaded timetable and captured JSONL files;
- `railway.db` and every table inside it;

- validation and recovery CSV files;
- timestamped HTML report folders and Downloads ZIP packages.

Credentials, connection state, page selection, and parameter controls are not persisted by the current version. Parameters return to their defaults after restart, although previously calculated tables remain available through the relevant **Load existing** button.

The selected display mode *is* persisted through Qt settings. The next launch restores either *Broadcast Control* or *Transit OS 2000*.

Application shell and common behaviour

5.1 Navigation and status

The left sidebar changes pages. The top-right header reports whether the credential test most recently succeeded in the current session. This indicator does not continuously monitor the data services and does not prove that a live STOMP subscription is active. The bottom status bar contains the latest operation message.

5.2 Display modes

The header's *DISPLAY MODE* selector changes the application stylesheet:

Broadcast Control

The default high-contrast operations-terminal presentation in the audited source snapshot.

Transit OS 2000

The pale blue transport-computing presentation shown in the project's earlier interface captures.

Changing display mode does not change data, thresholds, calculations, or output paths. The choice is saved for later sessions.

Long-running work executes in a background `QThread`. While a stage is running, its parameter and action controls are disabled. The page remains responsive, and the log or status bar receives progress messages.

5.3 How completion is indicated

A stage is finished when all of the following are true:

- the final success message appears in the page log or status bar;
- result cards/tables/charts have populated where applicable;
- disabled controls become active again.

An entry beginning `ERROR--` indicates termination due to a failure, not successful completion.

Connection page

6.1 Purpose and controls

The Connection page supplies three session values:

Username

Network Rail Open Data account email/username.

Password

Account password, masked by default. **Show** changes the field to plain text; **Hide** restores masking.

Data directory

Local workspace folder. **Browse** opens a folder chooser.

6.2 Testing credentials

1. Enter username and password.
2. Select the intended data directory.
3. Press **Test connection**.
4. Wait for *Credentials accepted* or a failure message.

The test sends an HTTPS GET request to the CIF authentication endpoint for the full daily timetable resource. HTTP success or redirect status codes are accepted; authentication failures and other HTTP statuses are reported.

Successful completion

The result text reads *Credentials accepted*; the header changes to *OPEN DATA CONNECTED*; and the status dot becomes green.

Important

The test does not download or parse the timetable, does not follow and verify the final redirected resource, and does not test the STOMP subscription. Use **Download timetable** on the Database page for the actual download. The header state applies only to the current session and the most recent accepted HTTP status.

6.3 Credential handling

The application does not deliberately save credentials to the database, configuration files, reports, or CSV exports. Worker objects clear their internal password strings after use. The password remains in the on-screen input field in process memory until the user clears it or exits the application. Do not commit credentials to Git, source code, screenshots, logs, or reports.

Live Feed page

7.1 Purpose

The Live Feed page captures live TRUST movement messages and stores each message as one JSON Lines record. It does not insert the messages into SQLite while capturing; importing is a separate Database-page operation.

7.2 Starting and stopping capture

1. Enter credentials on the Connection page.
2. Open Live Feed.
3. Press **Start capture**.
4. Confirm the state changes from *CONNECTING* to *CAPTURING*.
5. Leave the application running for the desired observation period.
6. Press **Stop capture** and wait for *STOPPED*.

The page displays message count, received batch count, elapsed time, output path, and a timestamped console log. Each new capture creates a new file; it does not overwrite earlier captures.

7.3 Technical connection details

The implemented worker uses STOMP 1.2 with:

Host	<code>publicdatafeeds.networkrail.co.uk</code>
Port	<code>61618</code>
Topic	<code>/topic/TRAIN_MVT_ALL_TOC</code>
Acknowledgement	<code>automatic</code>
Heartbeats	<code>10,000 ms in each direction</code>
Keepalive	<code>enabled</code>

An incoming STOMP body may contain one object or a list. For every contained message, the application writes

```
{"received_at_utc": "...", "message": {...}}
```

to a timestamped `.jsonl` file. The receipt time is recorded in UTC. Invalid JSON batches are skipped and logged. A disk-write error stops capture.

If this goes wrong

If no messages arrive, check the feed state and console before assuming the application has failed. Traffic volume varies. Authentication errors, STOMP errors, and unexpected disconnections appear in the console. Never terminate the terminal abruptly while a capture is active; use **Stop capture** so the connection closes cleanly.

Database page

8.1 Purpose and safe order of operations

The Database page converts raw files into the SQLite workspace used by every analytical page. Use the controls in this order:

1. **Download timetable**;
2. **Build timetable database**;
3. **Load movement file**;
4. **Refresh summary** whenever an external change has been made.

The summary cards report database size, schedule count, schedule-location count, and movement count. The displayed path is the exact `railway.db` currently being inspected.

8.2 Downloading the full timetable

This operation requires username and password. It requests the `CIF_ALL_FULL_DAILY` resource with day value `toc-full`, streams the response in 1 MiB chunks, and writes initially to `schedule_full.json.gz.part`. Only a complete download is renamed to `schedule_full.json.gz`; the temporary part file is removed after failure.

The progress bar is determinate when the server provides content length and indeterminate otherwise.

8.3 Building the timetable database

Pressing **Build timetable database** requires confirmation because the following tables are dropped and reconstructed:

- `tiploc`;
- `schedules`;
- `schedule_locations`.

The `train_movements` table is retained. The gzipped timetable is read line by line. `TiplocV1` records populate the location lookup, while `JsonScheduleV1` records populate one schedule and its ordered schedule locations. Work is committed every 5,000 input records and at completion.

Important

Because the build commits in batches, an interruption after one or more commits can leave partially rebuilt timetable tables. Treat any failed build as incomplete: preserve the log, check disk space and the source gzip, and rerun the complete build before analysis. A nonzero row count alone does not prove that a failed build completed.

The schedule and schedule-location rows are available for inspection and future development. The current downstream services do not join them to TRUST train IDs; only `tiploc` contributes to current location-name enrichment.

8.4 Loading a movement file

Load movement file opens a file chooser for a captured `.jsonl` file. Confirmation is required because the movement schema drops and recreates `train_movements`. It therefore *replaces*, rather than appends to, the previously imported capture.

For each record, the importer:

- extracts the envelope receipt time, message header, and body;
- converts millisecond timestamps to ISO-formatted UTC strings;
- converts numeric fields defensively;
- trims location, platform, and route text;
- preserves the raw message body as JSON;
- skips and counts lines that cannot be inserted.

Interpretation warning

Loading several separate capture files one after another does not combine them: the last imported file replaces the earlier movement table. If a combined analysis is required, combine and validate the JSONL records outside the application before importing, or extend the importer with a controlled append and deduplication workflow.

8.5 Core database schema

8.5.1 Timetable tables

Table	Contents
<code>tiploc</code>	TIPLOC code, transaction type, NALCO, STANOX, CRS, and two descriptions.
<code>schedules</code>	Internal schedule ID, CIF train UID, validity dates, days-run mask, bank-holiday flag, train status, STP indicator, and operator code.
<code>schedule_locations</code>	Ordered TIPLOC calls with arrival, departure, pass time, platform, line, and path. Keyed by schedule and location index.

8.5.2 Movement table field groups

Group	Stored columns
Receipt/header	<code>received_at_utc</code> , queue timestamp in ms and UTC, message type, original source, source system.
Identity/event	<code>train_id</code> , event type, planned event type, event source.
Location	current STANOX, reporting STANOX, next reporting STANOX, next-report run time.
Time	planned, actual, and GBTT timestamps in both milliseconds and UTC text.
Delay	timetable variation and variation status.
Movement context	direction, platform, route, train service code, division, operator.
Flags	terminated, delay-monitoring point, auto-expected, correction, and off-route indicators.
Raw evidence	complete message body JSON.

Indexes are created on train ID, event type, location STANOX, actual time, and planned time to support the later queries.

Successful completion

The database stage is ready when the timetable and movement counts are nonzero, the operation log contains no final error, and the application reports the successful build/import message. Exact counts depend on the downloaded timetable and length of the selected capture.

Train Analysis page

9.1 Purpose and prerequisites

This page reconstructs one observed train’s ordered movement sequence and plots how its stored timetable-variation value changes through the captured journey. A populated `train_movements` table and timetable location lookup are required.

9.2 Selecting a train

1. Press **Refresh candidates**.
2. Use the visible down-arrow to open the candidate list, or type an exact train ID into the editable field.
3. Press **Analyse train**, or press Enter in the train-ID field.

The candidate query returns at most 100 trains, ranked first by descending event count and then by train ID. A train absent from this top-100 list can still be analysed by entering its exact ID.

9.3 Event reconstruction

Events for the chosen train are ordered by `actual_time_utc`. The location name is resolved by matching movement STANOX to the timetable lookup and taking the TPS description, then the general description, then the raw STANOX, and finally “Unknown location” as fallbacks.

For event index j , the application defines

$$\Delta V_j = V_j - V_{j-1}. \tag{9.1}$$

The first event has no previous variation and therefore no displayed change.

9.4 Metric cards and classification

Let $V_1, \dots, V_n$ be the non-missing variations in observed time order. The cards show

$$\begin{aligned} \text{events} &= \text{number of observed rows,} \\ \text{starting variation} &= V_1, \\ \text{ending variation} &= V_n, \\ \text{net change} &= V_n - V_1. \end{aligned}$$

The service also computes the maximum variation and numbers of positive, negative, and zero changes.

Let

$$J_{\max} = \max_{j \geq 2} \Delta V_j, \quad N_+ = \#\{j : \Delta V_j > 0\}. \quad (9.2)$$

The displayed case label uses the following deterministic rules, evaluated in order:

Case label	Implemented condition
SUDDEN DISRUPTION	$J_{\max} \geq 10$ and $N_+ \leq 2$.
PROPAGATION	$N_+ \geq 2$, $V_n - V_1 > 0$, and $J_{\max} < 10$.
MIXED	$N_+ \geq 2$, $V_n - V_1 > 0$, and $J_{\max} \geq 10$, unless the preceding sudden-disruption rule already matched.
STABLE OR RECOVERING	Every remaining observed pattern.
NO DATA	No usable event rows.

Interpretation warning

These labels are descriptive heuristics. “Propagation” on this page means a train accumulated delay through multiple increases; it does not identify which other train caused those increases. Train-to-train inference occurs on the Propagation page.

9.5 Chart and event table

The turquoise line plots stored timetable variation against actual UTC time. The horizontal zero line is a numeric reference; in the audited build it must not be assumed to separate late from early movements because status is not converted into a sign. The table shows actual time, resolved location, event type, variation, change, feed status, and platform. Missing values are displayed as an em dash.

Known analytical limitation

Always inspect the *Status* column alongside the plotted numeric values. Until variation-status normalisation is implemented, the chart is a plot of the stored feed value, not a verified signed delay trajectory.

Propagation Analysis page

10.1 Purpose

The Propagation page converts close-time, same-location movement sequences into a directed, weighted network of candidate delay transmission. It first works at event-pair level, then aggregates high-confidence pairs into train-to-train edges, train nodes, and location summaries.

10.2 Parameters

Control	Default	Meaning
Maximum gap	10 min	Largest permitted elapsed time from source event to affected event.
Source delay	5 min	Minimum source-train variation for candidate status.
Affected delay	5 min	Minimum affected-train variation at interaction.
Causal score	8	Minimum composite score for high-confidence status.
Require same direction	selected	Rejects pairs whose direction indicators differ.

Changing a value does nothing until **Run propagation analysis** is pressed. **Load existing results** ignores the displayed controls and loads the most recently saved tables.

10.3 Step 1: location enrichment

The service rebuilds `stanox_lookup` by grouping nonblank timetable STANOX values and choosing the minimum TPS description for each code. It then rebuilds `train_movements_enriched` by left-joining that lookup onto every movement. Rows without a matched readable location remain in the enriched table but are excluded from pair construction.

10.4 Step 2: constructing temporally ordered pairs

Within each location STANOX, usable events are ordered by actual time. A source event s and later affected event a are retained when

$$0 \leq \Delta t = t_a - t_s \leq G, \quad (10.1)$$

the train IDs differ, and—when selected—the direction indicators agree. Here G is the maximum-gap control. The implementation permits equal timestamps at this initial pairing stage.

10.5 Step 3: affected-train sequence context

For each affected train, the service orders its complete usable event sequence and attaches the previous and next events around the candidate interaction. The pair survives only when the affected interaction can be matched by both actual timestamp and STANOX. The quantities W_{pre} , W_{post} , and W_{inc} are then calculated as defined in Chapter 2.

10.6 Step 4: candidate and score construction

Let D_s and D_a be the selected source and affected thresholds. The basic candidate indicator is

$$C = \mathbf{1}\{V_s \geq D_s\} \mathbf{1}\{V^0 \geq D_a\}. \quad (10.2)$$

Define

$$S_s = [V_s]_+, \quad (10.3)$$

$$B_g = [10 - \Delta t]_+, \quad (10.4)$$

$$T_d = \min([V_s]_+, [W_{\text{post}}]_+). \quad (10.5)$$

The implemented composite score is

$$\begin{aligned} S = & 0.6S_s + 1.8[W_{\text{post}}]_+ + 1.2[W_{\text{inc}}]_+ + 0.4B_g \\ & + 0.8T_d + \mathbf{1}\{W_{\text{inc}} > 0\} - 0.6[W_{\text{pre}}]_+. \end{aligned} \quad (10.6)$$

Important

The short-gap bonus in Equation (10.6) uses the fixed constant 10, not the on-screen maximum-gap value. Increasing the maximum gap may admit pairs beyond ten minutes, but those pairs receive zero short-gap bonus.

Missing pre/post changes are filled with zero for scoring. A pair is labelled high confidence precisely when

$$H = C \mathbf{1}\{W_{\text{post}} > 0\} \mathbf{1}\{W_{\text{inc}} > 0\} \mathbf{1}\{S \geq S_{\text{min}}\} = 1. \quad (10.7)$$

Thus both positive post-interaction worsening and post-worsening stronger than pre-worsening are mandatory.

10.7 Step 5: directed edge aggregation

All high-confidence pairs sharing source u and affected train v are aggregated into edge $u \rightarrow v$. The edge stores event count, distinct-location count, mean gap, maximum source variation, mean and maximum post-worsening, mean incremental worsening, maximum/mean score, a sample location, and first/last interaction times.

Its ranking weight is

$$E_{uv} = N_{\text{events}} + 0.7N_{\text{locations}} + \overline{W}_{\text{post}} + 0.8\overline{W}_{\text{inc}} + 0.2V_{s,\text{max}} - 0.2\overline{\Delta t}. \quad (10.8)$$

10.8 Step 6: node and location summaries

For each train, outgoing and incoming degrees, weights, and event counts are summed. Total degree and total weight are the corresponding incoming plus outgoing values. Node roles are:

- **SOURCE ONLY:** positive outgoing weight and zero incoming weight;
- **SINK ONLY:** positive incoming weight and zero outgoing weight;
- **INTERMEDIATE:** positive weight in both directions;
- **ISOLATED:** neither, although isolated trains do not normally arise from the edge-derived node set.

At each location, the application calculates

$$L = N_{\text{pairs}} + 0.5N_{\text{affected}} + \overline{W}_{\text{post}} + 0.8\overline{W}_{\text{inc}} - 0.2\overline{\Delta t}. \quad (10.9)$$

10.9 Metrics and result tabs

Display	Interpretation
Evaluated pairs	Same-location close-time pairs successfully linked to affected-train sequence context.
High-confidence pairs	Individual pair-events satisfying all high-confidence rules.
Directed edges	Unique source–affected train relations after aggregation.
Network trains	Unique trains participating in at least one stored edge.
Strongest edges	Up to 200 edges ordered by edge weight.
Important locations	Up to 200 locations ordered by location score.
Pipeline log	Ordered construction and error messages.

10.10 Network explorer

The default display is a readable connected high-weight subnetwork. Selection starts with the globally strongest edge, then repeatedly chooses the strongest remaining edge that touches a

train already selected, up to 15 edges. It may stop earlier if no remaining edge touches the selected component.

Node colours mean:

- pink: source-only;
- blue: intermediate;
- turquoise: affected/sink-only.

Arrow direction is source to affected, and thicker arrows have greater edge weight. Node positions come from a force-directed spring layout and are not geographical.

Use the mouse wheel or **Zoom in/Zoom out**. Hold the left mouse button and drag to traverse; **Reset view** restores the fitted view. **Expand network** opens the same curated subnetwork in a maximised window, with an explicit legend. It does not replace the display with the entire raw network.

10.11 Completion and interpretation

Successful completion

Completion is indicated by *Propagation analysis results ready*, populated metrics/tabs, re-enabled controls, and final log messages confirming that the tables were saved and loaded into the explorer.

Interpretation warning

The coefficients in Equations (10.6)–(10.9) are heuristic modelling choices, not statistically fitted probabilities. A high score or heavy edge does not represent a probability of causation. Co-location, common infrastructure failures, congestion, timetable structure, or unobserved third trains may explain the same pattern.

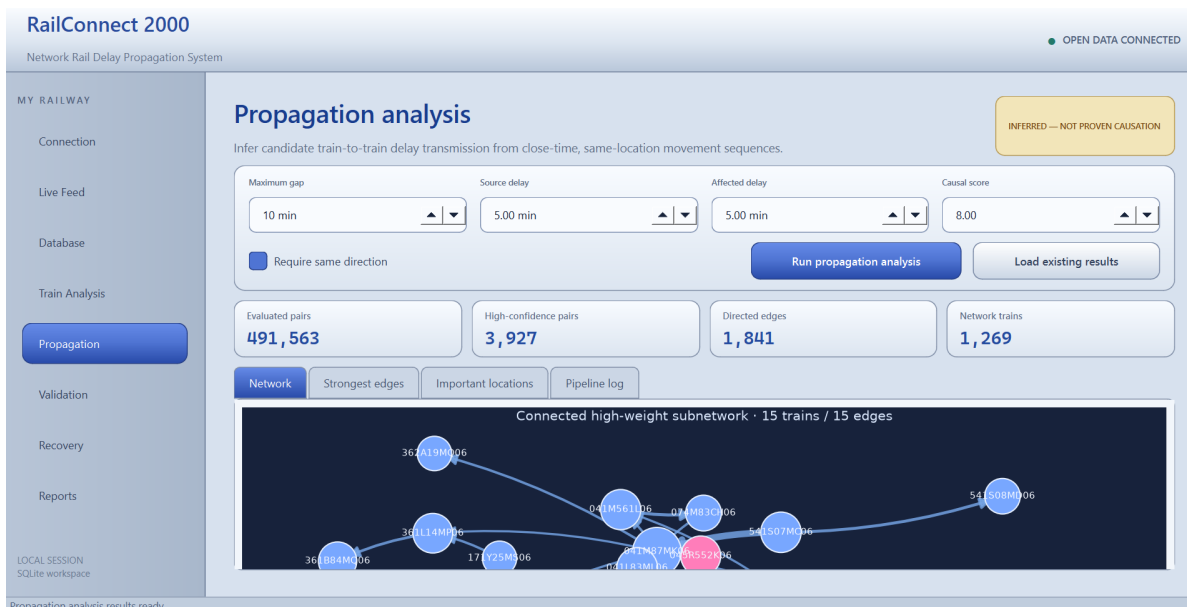


Figure 10.1: Propagation page in Transit OS 2000 mode. The visible network is a curated high-weight subnetwork; arrow direction and colour follow the legend described above. The numerical values are illustrative of one captured dataset and are not expected defaults.

Validation Laboratory

11.1 Purpose and prerequisites

Validation asks two distinct questions:

1. Do high-weight propagation cases look stronger than nearby local comparators that do not satisfy the same candidate rule?
2. Do broad propagation statistics remain similar when thresholds or plausibility conditions are changed?

It requires `train_movements_enriched` and `causal_propagation_edges`; therefore `Propagation` must run first.

11.2 Controls

Control	Default	Meaning
Maximum cases	50	Number of top propagation edges considered for matched-control validation.
Maximum interaction gap	10 min	Largest positive gap used to reconstruct the case interaction.
Control time window	30 min	Symmetric time interval around the affected event in which local controls are sought.

Run validation tests replaces previous validation tables. **Load existing results** only reads the last saved run.

11.3 Matched-control method

11.3.1 Sequence preparation

All usable enriched events are ordered within train. For every event the service computes previous variation, next variation, pre-change, post-change, and

$$Y = \mathbf{1}\{W_{\text{post}} > 0\}. \quad (11.1)$$

11.3.2 Reconstructing a propagation case

For each of the top K edges, the service searches the edge's sample location for source and affected events satisfying

$$0 < \Delta t \leq G \quad (11.2)$$

and equal direction indicators. If several pairs satisfy the rule, the smallest positive gap is selected. An edge for which no such event pair can be reconstructed is omitted, which is why *Cases evaluated* can be less than *Maximum cases*.

Known analytical limitation

This step does not retrieve the exact pair-event that originally caused the edge to be stored. An edge retains only a sample readable location, not the original pair row or sample STANOX. Validation searches again by source train, affected train, readable `location_name`, direction, and time gap, and chooses the smallest positive gap. It does not reapply the propagation source/affected thresholds, positive post/incremental-change tests, or score threshold to that reconstructed event.

The reconstructed case can therefore be a different encounter between the same trains and can fail the original high-confidence pair rule. The matched- control output is a diagnostic of that reconstructed encounter, not a formal validation of the exact pair-events aggregated into the edge.

11.3.3 Building the local control pool

For a case affected at time t_a , location ℓ , and direction d , initial control candidates are all event rows satisfying

$$\ell_c = \ell, \quad d_c = d, \quad t_a - W \leq t_c \leq t_a + W, \quad (11.3)$$

excluding the case source and affected trains. Here W is the control-window value.

A train can contribute more than one admissible event row. Thus `n_controls` counts control events, not necessarily distinct control trains.

The service then removes a potential control when it followed a different train within the preceding G minutes whose timetable variation was at least five minutes. The five-minute delayed-prior threshold is fixed in the implementation and is not controlled by the Validation-page fields. In the audited build it is applied to the raw numeric variation without consulting `variation_status`; the limitation in Chapter 2 therefore applies here as well.

For the remaining n_c control rows, let

$$\mathcal{C}^* = \{r : W_{\text{post},r} \text{ is non-missing}\}, \quad n_c^* = |\mathcal{C}^*|.$$

The service calculates

$$\hat{p}_c = \frac{1}{n_c} \sum_{r=1}^{n_c} Y_r, \quad (11.4)$$

$$\overline{W}_c = \frac{1}{n_c^*} \sum_{r \in \mathcal{C}^*} W_{\text{post},r}, \quad n_c^* > 0. \quad (11.5)$$

This distinction is required by the implementation: a missing next event makes `post_worsened` zero and therefore contributes zero to $\hat{p}_c$, whereas pandas excludes the same missing post-change from $\overline{W}_c$. If no control has a non-missing post-change, $\overline{W}_c$ is missing and the stronger-than-control comparison is false.

11.3.4 Classification rules

Let Y_a and W_a be the candidate affected event's worsening indicator and post-change. The displayed category is assigned in this exact order:

Result	Condition
NO_CONTROL_AVAILABLE	$n_c = 0$.
SUPPORTS_CASE	$n_c > 0$, $Y_a = 1$, and $W_a > \overline{W}_c$.
PARTIAL_SUPPORT	$n_c > 0$ and $Y_a = 1$, but the preceding stronger-than-control condition is false.
DOES_NOT_SUPPORT	$n_c > 0$ and $Y_a = 0$.

This is a transparent rule-based comparison. It is not a hypothesis test, p-value, confidence interval, matched-regression estimator, or causal effect estimate.

11.4 Threshold-sensitivity grid

Sensitivity testing reconstructs same-direction pairs for every combination

$$G \in \{5, 10, 15\} \text{ minutes}, \quad D \in \{3, 5, 10\} \text{ minutes}. \quad (11.6)$$

Both source and affected variations must be at least D . Nine settings are therefore produced. For each setting the table records:

- pair count and post-worsened count;
- proportion post-worsened;
- count and proportion with $W_{\text{post}} > W_{\text{pre}}$;
- mean and median post-change;
- unique locations, source trains, and affected trains;
- five most frequent locations.

For example,

$$\hat{p}_{\text{worse}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{W_{\text{post},i} > 0\}. \quad (11.7)$$

11.5 Negative and stress tests

The three fixed tests use $G = 10$ and $D = 5$:

same_direction

Positive gaps no greater than 10 minutes and matching directions.

opposite_direction

Positive gaps no greater than 10 minutes and differing directions.

large_gap

Gaps strictly greater than 10 and no greater than 30 minutes. The implemented large-gap mode does not additionally filter direction.

The expectation is not that every negative statistic must be zero. Rather, the same-direction immediate rule should ideally exhibit a more coherent signal than deliberately less plausible alternatives. Similar results across all modes warn that the analysis may be measuring broad congestion or background delay rather than immediate train-to-train transmission.

11.6 Reading the page

The five cards count evaluated, supporting, partially supporting, non-supporting, and no-control cases. The Overview tab charts matched-control outcomes and the three post-worsening rates. The remaining tabs expose the full matched-control rows, nine sensitivity summaries, three negative-test rows, and progress log.

Successful completion

The run has finished when the log confirms three validation tables were saved, the status bar reads *Validation results ready*, the controls are enabled, and the Sensitivity and Negative Tests tabs contain nine and three rows respectively.

11.7 Validation outputs

The SQLite tables are `validation_control_results`, `validation_threshold_sensitivity`, and `validation_negative_tests`. CSV counterparts are written to `outputs/tables`; the matched-control CSV is named `validation_control_comparison.csv`.

Recovery Optimisation

12.1 Purpose and warning

Recovery converts the propagation network into candidate train-level and location-level priorities, then solves a constrained binary selection problem. It requires nonempty `causal_propagation_edges`.

Interpretation warning

The model's benefit, effectiveness, and cost are dimensionless heuristics. They are not minutes of delay saved, pounds sterling, staff-hours, or measured intervention effects. The selected plan is a research demonstration of operations-research structure, not an instruction to railway controllers.

12.2 Controls

Control	Default	Meaning
Total budget	8.00 units	Maximum sum of heuristic intervention costs.
Maximum interventions	5	Cardinality limit on selected binary decisions.
Assumed effectiveness	0.50	Fraction of aggregated edge weight assumed avoidable.
Candidates per type	100	Highest-ranked train candidates and highest-ranked location candidates admitted before optimisation.

The total candidate count can be twice the per-type value when enough positive train and location candidates exist.

12.3 Train-level ranking

For source train u , let

$$\begin{aligned} A_u &= \text{number of distinct affected trains,} \\ L_u &= \text{number of distinct sample locations,} \\ T_u &= \sum_v E_{uv}, \\ e &= \text{assumed effectiveness.} \end{aligned}$$

The estimated avoided impact and priority score are

$$b_u = eT_u, \quad (12.1)$$

$$P_u = b_u + 0.5A_u + 0.2L_u. \quad (12.2)$$

Trains are ranked primarily by P_u , then by affected-train count.

12.4 Location-level ranking

For location ℓ , let S_ℓ and A_ℓ be its distinct source and affected counts, and T_ℓ its sum of edge weights. Then

$$b_\ell = eT_\ell, \quad (12.3)$$

$$P_\ell = b_\ell + 0.4S_\ell + 0.4A_\ell. \quad (12.4)$$

12.5 Candidate cost and binary programme

For either candidate i , with affected count A_i , the heuristic cost is

$$c_i = 1 + 0.25A_i. \quad (12.5)$$

Let $x_i \in \{0, 1\}$ indicate selection, B be total budget, and M the maximum number of interventions. The model is

$$\max_x \quad \sum_i b_i x_i, \quad (12.6)$$

$$\text{subject to} \quad \sum_i c_i x_i \leq B, \quad (12.7)$$

$$\sum_i x_i \leq M, \quad (12.8)$$

$$x_i \in \{0, 1\} \quad \text{for every } i. \quad (12.9)$$

The priority scores rank and describe candidates but the binary objective maximises estimated benefit b_i , not P_i . No explicit mutual-exclusion constraint prevents selecting a train and a location that overlap operationally.

12.6 Exact dynamic-programming solver

Because Equation (12.5) uses quarter units, costs and budget are multiplied by four and rounded to integers. The solver maintains states (q, m) , where q is used quarter-budget and m selected

count, together with the greatest benefit and corresponding subset known for that state. Each candidate generates skip and take transitions; transitions exceeding either constraint are discarded. The maximum-benefit surviving state is exact for the integerised model. No SciPy or external MILP solver is required.

The number selected may be below M because the budget can bind first.

Important

Candidate costs are exact quarter units, but the on-screen budget control can accept values between quarter units. The solver uses $\text{round}(4B)/4$ internally. For an unambiguous experiment, enter budgets in increments of 0.25 units and record the entered and used budgets. This is integerisation of a heuristic model, not a conversion to currency or operational resource units.

12.7 Reading the page and outputs

Cards report admitted candidates, selected interventions, used budget, and total estimated avoided impact. The Selected Plan chart displays selected benefits; blue bars are trains and turquoise bars are locations. The other tabs show the complete candidate solution, train rankings, location rankings, and optimisation log.

The run replaces these SQLite tables:

- `recovery_intervention_plan_by_train`;
- `recovery_intervention_plan_by_location`;
- `recovery_intervention_solution`;
- `recovery_intervention_summary`.

Four corresponding CSVs are placed in `outputs/tables`; the solution and summary filenames retain the project's historical `bip` wording.

Successful completion

Completion is indicated by *Recovery optimisation results ready*, a populated selected-plan chart, re-enabled controls, and a log message reporting the number selected and budget used.

Reports and Exports

13.1 Inventory

The Reports page inspects the current `railway.db` and marks each supported analysis section as *READY* or *NOT YET AVAILABLE*. It also counts movement events, unique trains, propagation edges, validation cases, and selected recovery actions when those source tables exist.

Refresh report inventory repeats this read-only inspection. Missing sections do not prevent a partial report: every available supported table is included.

13.2 Generating an HTML report

Generate HTML report creates a new timestamped folder beneath `outputs/reports`. The HTML document contains summary cards, an interpretation warning, and a preview of the first 20 rows of each available table. Each table preview has its own horizontal scrollbar. Sticky headers and alternating shading improve readability.

The full rows—not merely the 20-row previews—are exported as CSV files in the report’s `tables` subfolder. Supported sources are:

- propagation edges and locations;
- validation controls, sensitivity, and negative tests;
- recovery train plan, location plan, solution, and summary.

Open latest report opens the latest report generated during the current application session in the default browser.

13.3 Exporting to Downloads

After a report has been generated, **Export to Downloads** creates a ZIP containing the HTML document and complete CSV table folder. The operating system’s standard Downloads location is used. An existing file is never overwritten; later copies receive suffixes such as `-copy-2`.

Important

The Open and Export buttons refer to the latest report generated in the current session. After restarting the application, generate a fresh report before using them. Existing report

folders remain on disk and can always be opened manually in File Explorer.

13.4 Report security and interpretation

Credentials are not included. Reports may nevertheless contain train IDs, locations, movement-derived statistics, and model results. Treat the package as project data, review it before public distribution, and preserve the interpretation warning when sharing results.

Complete End-to-End Workflow

This chapter is the recommended operating procedure for a new dataset. Follow the sequence exactly on the first run. Later runs may reuse timetable and movement data, provided the operator understands which results will be replaced.

14.1 Phase 1: start and establish the workspace

1. Open a terminal in the project root, activate `.venv`, and run `python-mdesktop_app.main`.
2. On Connection, enter the Network Rail Open Data username and password.
3. Choose the project `data` directory. Confirm that the displayed path ends in `data`, not the project root or `raw` folder.
4. Select **Test connection**. Continue only after *Credentials accepted* appears.

The credential check and the workspace selection are separate ideas. An accepted result means the endpoint returned one of the response codes treated as success by the application; it is not proof of a completed download or live STOMP connection. The data directory tells every page where local data will be read and written.

14.2 Phase 2: acquire source data

Timetable

On Database, select **Download full timetable**. Wait for the activity log to confirm that `schedule_full.json.gz` has been saved. This file is large and the operation can take time. A temporary `.part` file during download is normal.

Movements

On Live Feed, select **Start capture**. Leave the application running during the observation period. The message count should rise when the feed is active. Select **Stop capture** before loading that file into SQLite. The captured filename shown by the page is the file to select later.

Important

The current application uses the timetable's TIPLOC records for readable location enrichment and the movement messages for analytical event sequences. It stores full schedules but does not currently match them to TRUST journeys. A very short live capture may contain too few repeated trains or same-location interactions for propagation analysis.

14.3 Phase 3: build the local database

On Database:

1. Select **Build timetable database** and wait for completion.
2. Select **Load movement file** and choose one stopped `train_movements_*.jsonl` capture.
3. Verify that the four cards show a plausible database size and nonzero counts for *Schedules*, *Schedule locations*, and *Movements*. TIPLOC and distinct-train counts are not displayed as Database-page cards in the audited build.

Loading a movement file replaces the existing `train_movements` table. It is therefore a deliberate dataset selection, not an append operation.

14.4 Phase 4: inspect data quality before modelling

Use Train Analysis before running the larger pipeline:

1. Select **Refresh candidates**.
2. Choose several high-event-count train IDs and select **Analyse train**.
3. Check that event times are ordered, location names are present, and raw variation values are consistent with the accompanying *Status* values.

This check catches truncated captures, missing location matches, and unexpected variation fields before the more computationally expensive stages.

14.5 Phase 5: construct the propagation network

On Propagation, begin with the defaults unless there is a documented reason to change them. Select **Run propagation analysis**. Do not close the application while it is running. Completion requires populated cards, re-enabled controls, final log messages, and the status *Propagation analysis results ready*.

Inspect all four tabs. In particular:

- use Network to understand topology, never geography;
- use Strongest edges to inspect the measurements behind an arrow;
- use Important locations to identify repeated interaction sites;
- use Pipeline log to retain a record of counts and any warnings.

14.6 Phase 6: validate the inferred relationships

On Validation, select **Run validation tests**. The page uses existing propagation edges; it does not create an independent causal model. Inspect the balance among Supports case, Partial support, Does not support, and No control. Then compare same-direction, opposite-direction, and large-gap negative tests.

Do not report only the number of supported cases. Also report the number evaluated, cases without controls, the control-window setting, and the fixed sensitivity/negative-test definitions documented in Chapter 11.

14.7 Phase 7: explore a constrained recovery plan

On Recovery, enter a hypothetical budget, intervention limit, effectiveness, and candidate limit. Select **Optimise recovery plan**. Treat the output as a ranking experiment: the displayed benefit and budget values are dimensionless model units. They are not predicted delay minutes, monetary costs, or validated operational actions.

14.8 Phase 8: generate and preserve the report

On Reports:

1. Select **Refresh report inventory** and review missing sections.
2. Select **Generate HTML report**.
3. Select **Open latest report** and visually inspect every section.
4. Select **Export to Downloads** to create a portable ZIP containing the report and complete CSV tables.

Keep the ZIP together with a note recording the source capture filename and the propagation, validation, and recovery settings. The current database does not persist every run parameter as a formal experiment record.

14.9 End-to-end acceptance checklist

Check	Required	Evidence
Application starts	Yes	Main window and all navigation pages open.
Credential test passes	For download-s/feed	Green connection indicator and accepted message.
Timetable file complete	Yes	Gzip file exists; no lingering .part file after success.
Live capture complete	Yes	Stopped JSONL file with non-zero messages.

Check	Required	Evidence
Database populated	Yes	Database size is plausible; schedule, schedule-location, and movement cards are nonzero.
Train sequences plausible	Yes	Ordered events, resolved locations, and variation/status combinations have been inspected.
Propagation completed	Yes	Four result tables and non-empty log; edges may legitimately be zero under strict settings.
Validation completed	Recommended	Three validation tables and outcome counts.
Recovery completed	Optional model stage	Four recovery tables and a solver summary.
Report reviewed/exported	Yes for sharing	HTML opens and Downloads contains the timestamped ZIP.
Interpretation warning retained	Yes	Report/manual caveat accompanies any communicated finding.

Empirical Case Study

15.1 Purpose and evidence boundary

This chapter applies the complete workflow to the operator’s day-long movement capture associated with 6 June 2026. It is included to demonstrate what the pipeline produces at realistic scale and how its outputs should be interpreted. It is not presented as a causal-impact study or an operational performance assessment.

The numerical source used here is the latest supplied desktop-application report, `railconnect-report-20260902-070325`. Its complete CSV tables are byte-identical to the tables in the two earlier reports generated at 06:53 and 07:00 on the same morning; those three folders are repeated exports of one analysis state rather than independent replications. The HTML report was generated on 2 September 2026, while every exported propagation and validation interaction timestamp lies on 6 June 2026.

Important

The supplied case-study archive does not contain the original movement JSONL, the capture log, or `railway.db`. The operator identifies the source as a day-long capture, but its exact start time, stop time, message-receipt count, and continuity cannot be independently reconstructed from the report package. The edge table’s high-confidence interactions span 07:30–17:27 UTC; that is the span of retained interactions, not proof of the raw capture duration.

The archive also contains earlier June summaries produced by predecessor scripts. Where those summaries differ from the desktop report, this chapter uses the desktop report. In particular, the legacy summary names `scipy_milp`, whereas the current report records `exact_dynamic_programming`. Mixing those two result generations would produce an internally inconsistent case study.

15.2 Recorded configuration

The screenshots and report tables identify the following settings:

Stage and control	Value	Role
Propagation: maximum gap	10 min	Largest admitted source-to-affected gap.
Propagation: source threshold	5	Minimum raw source-variation value.

Stage and control	Value	Role
Propagation: affected threshold	5	Minimum raw affected-variation value.
Propagation: score threshold	8	Minimum heuristic score for high-confidence status.
Propagation: same direction	Required	Equal direction indicators required.
Validation: maximum cases	50	Highest-weight edges attempted; 48 were reconstructable.
Validation: interaction gap	10 min	Reconstruction gap limit.
Validation: control window	30 min	Symmetric local-control window.
Recovery: budget	8 units	Maximum dimensionless intervention cost.
Recovery: maximum interventions	5	Cardinality cap.
Recovery: effectiveness	0.50	Heuristic multiplier applied to edge weights.
Recovery: candidates per type	100	Up to 100 train and 100 location candidates.

As throughout the application, each threshold uses the stored numeric `timetable_variation` value without signing it from `variation_status`. The values below are therefore analysis variables measured in feed-reported minutes, not verified signed lateness.

15.3 Scale and analytical funnel

Quantity	Result	Evidence
Loaded movement events	402,270	HTML report inventory.
Distinct movement train IDs	18,016	HTML report inventory.
Evaluated same-location pairs	491,563	Propagation-page result card in Figure 10.1.
High-confidence pair-events	3,927	Propagation card; independently equals the sum of <code>n_causal_events</code> .
Directed aggregate edges	1,841	Edge-table row count.
Trains in the inferred network	1,269	Unique union of source and affected IDs.
Locations with retained pairs	596	Location-table row count.

Only $3,927/491,563 \approx 0.80\%$ of evaluated pairs reached the high-confidence rule. Those events were aggregated into 1,841 directed train-to-train relations. The median edge contains one retained event and 922 edges—50.1% of all edges—contain exactly one. The network is therefore large but evidentially sparse at the individual-edge level; an extreme weight can be driven by one extreme event rather than repeated encounters.

15.4 Location-level structure

The highest location scores were:

Rank	Location	Pairs	Sources	Affected	Score
1	Manchester Piccadilly	162	87	53	190.30
2	Manchester Oxford Road	130	61	40	151.71
3	Stockport	104	50	35	123.37
4	Slade Lane Junction	65	57	36	86.63
5	Tring	47	37	31	63.59

This ordering identifies locations at which the heuristic repeatedly found close-time interactions with subsequent variation changes. It does not mean that these locations caused delay, nor does it adjust for traffic volume, service mix, timetable density, route topology, or common disruption exposure. The scores are useful for prioritising investigation, not for comparing the operational performance of stations or junctions.

15.5 Edge ranking versus validation

Figure 15.1 deliberately combines two stages that should not be conflated. Bar length is the propagation edge weight; colour is the later reconstructed-case validation result.

The highest-weight edge is 045R552K06 to 041M87MK06 at Edinburgh. Its edge row records weight 110.70, one retained pair-event, source variation 32, mean post-worsening 43, mean incremental worsening 76, and maximum pair score 216. On those propagation quantities it ranks first.

Validation then reconstructs an Edinburgh encounter for the same train IDs. It finds a six-minute gap and two local control events, but the reconstructed affected event has post-change zero, producing DOES_NOT_SUPPORT. The second- and third-ranked Edinburgh edges receive the same classification. This is not an arithmetic contradiction: as documented in Section 11.3.2, an aggregate edge retains a sample location rather than the exact originating pair identity, and validation searches again for the smallest positive-gap encounter.

Interpretation warning

This worked example exposes a material traceability weakness. A future schema should preserve a stable identifier for every contributing pair-event and make validation reference that identifier. Until then, a validation classification must be described as applying to a reconstructed encounter, not necessarily to the precise event that created the edge.

15.6 Validation and robustness results

Of 48 reconstructed cases:

- 32 (66.7%) were classified SUPPORTS_CASE;
- 4 (8.3%) were PARTIAL_SUPPORT;
- 7 (14.6%) were DOES_NOT_SUPPORT;

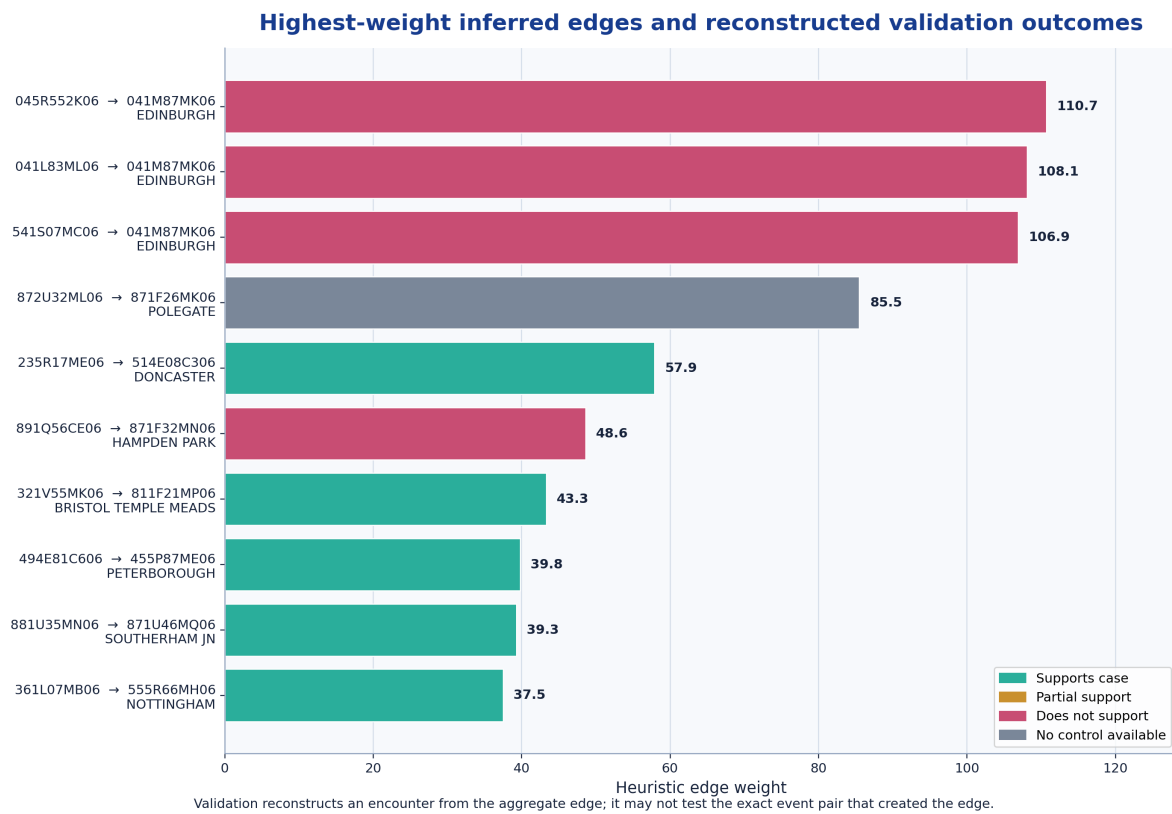


Figure 15.1: Ten highest-weight inferred edges, coloured by the validation classification of the reconstructed case. The validation service can test a different encounter from the event that generated the aggregate edge, so colour is not a direct adjudication of the bar's original event.

- 5 (10.4%) had NO_CONTROL_AVAILABLE.

Thus 36 of 48 cases received at least partial support, but this 75% figure is not an estimated accuracy or causal-confirmation rate. The cases are the highest-weight edges that could be reconstructed, not a random sample, and the classification is a deterministic comparison without uncertainty intervals.

Across the nine sensitivity settings, qualifying-pair counts ranged from 2,216 to 58,565 and the post-worsening rate ranged from 27.84% to 35.38%. At the baseline 10-minute gap and threshold 5, the same-direction rule produced 19,342 pairs and a 31.17% post-worsening rate. The opposite-direction test produced 27.56%, while the large-gap test produced 27.81%. The baseline was therefore higher by 3.61 and 3.36 percentage points respectively: a direction consistent with the heuristic, but a modest descriptive separation rather than causal identification.

15.7 Recovery optimisation

The current exact dynamic-programming run admitted 200 candidates. With budget $B = 8$, maximum interventions $M = 5$, and assumed effectiveness $e = 0.5$, it selected three location interventions:

Selected target	Heuristic cost	Estimated impact
Edinburgh	4.00	244.64
Polegate	2.50	93.05
Monktonhall Junction	1.50	68.37
Total	8.00	406.06

Manchester Piccadilly had the largest individual estimated impact, 446.47, but its heuristic cost was 14.25 and therefore exceeded the entire budget. The selected combination exactly exhausts the budget and illustrates the difference between ranking and constrained optimisation: the largest candidate need not belong to the best feasible subset. No train-level candidate was selected in the optimum for this parameterisation.

The impact and cost values remain dimensionless. The result does not predict 406 delay-minutes saved, and the place names do not constitute operational recommendations.

15.8 What the case study demonstrates

The run demonstrates that the application can move a large live-feed-derived event table through a reproducible analytical sequence: pair enumeration, strict heuristic filtering, network aggregation, matched-control diagnostics, sensitivity and negative tests, and exact constrained selection. It also demonstrates why the interpretation warnings are substantive rather than decorative:

- fewer than 1% of evaluated pairs pass the high-confidence rule;
- about half of aggregate edges rest on one retained event;
- the strongest aggregate edges need not receive supportive reconstructed validation outcomes;
- alternative thresholds materially change the admitted pair count;

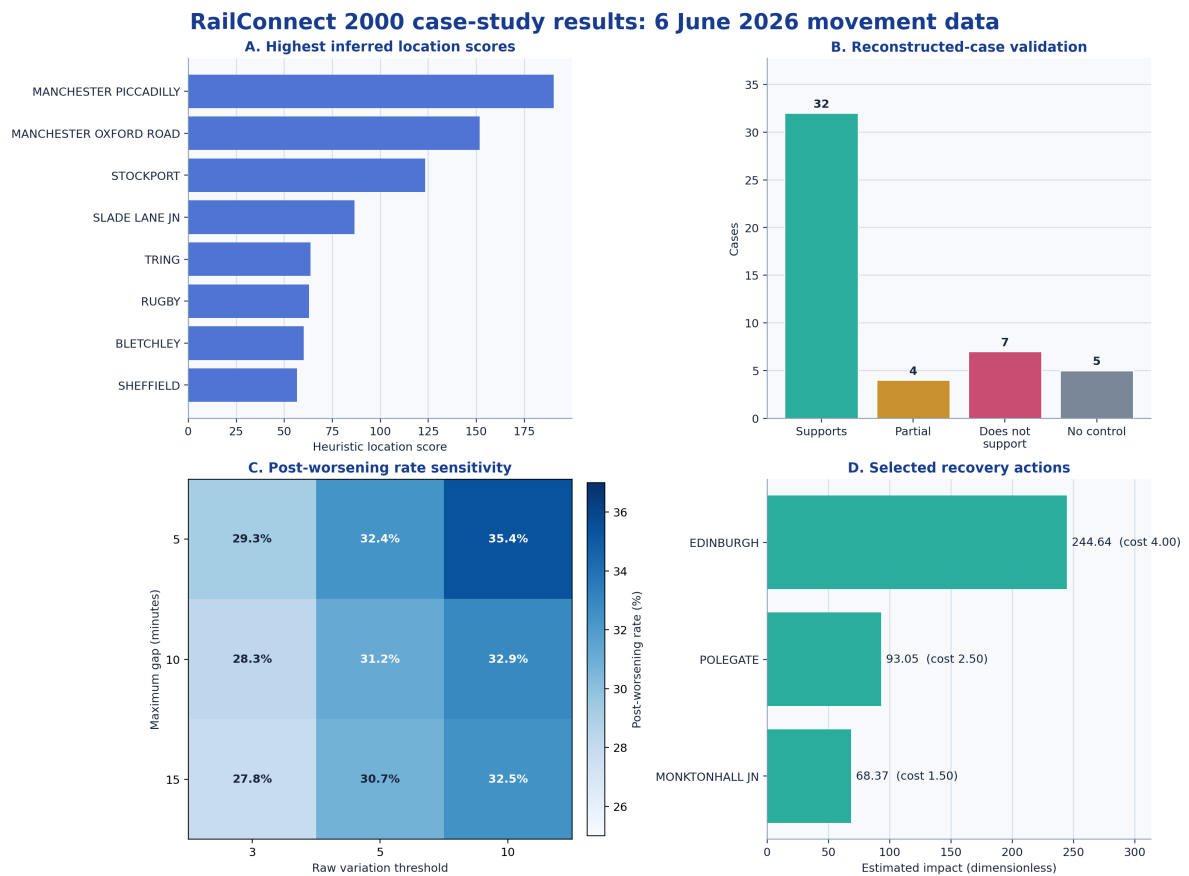


Figure 15.2: Case-study summary derived from the exported CSV tables. Panels show heuristic location scores, reconstructed validation outcomes, threshold sensitivity, and the selected recovery combination. None of the plotted quantities is a calibrated causal probability or operational effect.

- the optimiser is exact for its mathematical model, but the model’s inputs are heuristic and dimensionless.

A defensible portfolio statement is therefore:

“On a day-scale movement dataset containing 402,270 stored events, the pipeline evaluated 491,563 close-time same-location pairs, retained 3,927 high-confidence pair-events, aggregated them into 1,841 directed edges, ran reconstructed-case and robustness diagnostics, and solved an exact budget-constrained recovery-prioritisation model.”

It would not be defensible to replace “inferred,” “heuristic,” or “reconstructed” with a claim that the application proved which trains caused delay or quantified real intervention savings.

15.9 Reproducing this case study more completely

The supplied report ZIP is sufficient to reproduce the CSV-derived numbers and both new charts in this chapter. The evaluated-pair total is instead retained in the supplied propagation-page screenshot (Figure 10.1). A stronger audit package for a future run should additionally preserve:

- the stopped movement JSONL and its checksum;
- the corresponding `railway.db` snapshot;
- capture start/stop times, received-message count, and feed log;
- the Git commit hash and `pip freeze` output;
- every UI parameter and fixed internal constant;
- the generated report ZIP and a short interpretation record.

Data Lifecycle and Reproducibility

16.1 What each operation changes

Operation	Created or replaced	Preserved
Timetable download	<code>raw/schedule_full.json.gz</code> ; temporary <code>.part</code> while active	Database and captures
Build timetable database	<code>tiploc</code> , <code>schedules</code> , <code>schedule_locations</code>	<code>train_movements</code> and existing analysis tables
Load movement file	Entire <code>train_movements</code> table	Timetable tables and source JSONL files
Run propagation	Lookup/enriched tables and four <code>causal_propagation_*</code> tables	Source timetable and movements
Run validation	Three <code>validation_*</code> tables and CSVs	Propagation and source tables
Run recovery	Four <code>recovery_*</code> tables and CSVs	Propagation, validation, and source tables
Generate report	New timestamped report directory	All earlier reports and the database
Export report	New ZIP in Downloads; suffix added on collision	Source report directory

16.2 Why rerun order matters

Derived stages are snapshots of their inputs at run time. If movements are reloaded or propagation is rerun, old validation and recovery tables can still exist but describe the previous network until those stages are rerun. The application does not currently maintain dependency hashes or automatically invalidate downstream tables.

For a coherent experiment, use this rule:

changed source data $\implies$ rerun propagation
 $\implies$ rerun validation and recovery
 $\implies$ generate a new report.

16.3 Minimum experiment record

Record the following outside the database or in the exported report notes:

- timetable download date and movement capture filename/time window;
- application or Git commit/version;
- all four propagation controls and the direction checkbox;
- all three validation controls;
- all four recovery controls;
- generated report directory/ZIP name;
- any data cleaning, interruptions, or manual file changes.

Propagation and validation parameters are not stored as a dedicated run table. The recovery summary stores its principal run settings, but this alone is not a complete provenance record.

Database and Output Reference

17.1 SQLite table inventory

The following table is the authoritative functional inventory. SQLite may infer concrete storage types from imported values; the descriptions below therefore emphasise semantic meaning rather than a rigid external schema.

Table	Stage	Meaning
<code>tiploc</code>	Timetable	Location reference data, including TIPLOC, STANOX, CRS and description fields where supplied.
<code>schedules</code>	Timetable	One record per loaded CIF schedule with identifying and operating metadata.
<code>schedule_locations</code>	Timetable	Ordered planned calling/passing points belonging to schedules.
<code>train_movements</code>	Movements	Parsed TRUST movement events from the selected JSONL capture.
<code>stanox_lookup</code>	Propagation	One display name per non-empty STANOX, plus the number of source matches.
<code>train_movements_enriched</code>	Propagation	Movement columns plus resolved <code>location_name</code> and <code>stanox_match_count</code> .
<code>causal_propagation_pairs</code>	Propagation	Every evaluated same-location pair with affected-train sequence context, signals, score, and flags.
<code>causal_propagation_edges</code>	Propagation	High-confidence pairs aggregated by source and affected train.
<code>causal_propagation_nodes</code>	Propagation	Incoming/outgoing graph measures by train.
<code>causal_propagation_locations</code>	Propagation	High-confidence interaction summaries by location.
<code>validation_control_results</code>	Validation	One reconstructed propagation case per tested edge and its matched-control summary/classification.
<code>validation_threshold_sensitivity</code>	Validation	Summary results for the fixed nine-cell gap/delay grid.

Table	Stage	Meaning
validation_negative_tests	Validation	Same-direction, opposite-direction, and large-gap comparison summaries.
recovery_intervention_plan_by_train	Recovery	Ranked source-train candidates and selection flag.
recovery_intervention_plan_by_location	Recovery	Ranked location candidates and selection flag.
recovery_intervention_solution	Recovery	Combined candidate set with costs, benefits, and binary selection.
recovery_intervention_summary	Recovery	Single-row solver/run summary.

17.2 Movement-event fields

The user-visible and analysis-critical movement fields are:

Field	Meaning
train_id	TRUST train identifier used to group a movement sequence.
event_type	Arrival, departure, or other movement event type supplied by the feed.
loc_stanox	STANOX location code used for same-location matching.
actual_time_utc	Parsed actual event time in UTC.
planned_time_utc	Parsed planned event time in UTC where available.
timetable_variation	Feed-provided number of minutes' variation. The audited analysis uses the numeric value directly.
variation_status	Separate feed direction/status such as EARLY , ONTIME , LATE , or OFFROUTE ; stored but not used to sign the analysis value.
platform	Reported platform where supplied.
direction_ind	Direction indicator used by optional same-direction matching.
toc_id	Train operating company identifier.
raw_body_json	Original movement body retained for traceability.

Times are converted from feed millisecond timestamps to UTC during movement loading. A null or malformed value can make an event unusable for a later stage even if the raw row itself was stored.

17.3 Propagation-pair field groups

The pair table is deliberately wide. Its columns fall into six groups:

Group	Representative columns	Purpose
Interaction identity	<code>location_name</code> , <code>loc_stanox</code> , <code>direction_ind</code> , <code>time_gap_minutes</code>	Where and how closely the trains met.
Source event	<code>source_train_id</code> , event type, actual time, <code>source_variation</code>	Candidate upstream train state.
Affected event	<code>affected_train_id</code> , event type, actual time, variation at interaction	Candidate downstream train state at the meeting point.
Sequence context	previous/next affected time, location and variation; pre/post changes	Establishes local before/after behaviour of the affected train.
Signals	source strength, pre/post worsening, incremental worsening, short-gap bonus, transfer signal	Intermediate terms in Equation (10.6).
Decisions	candidate flag, post-worse flag, post-stronger flag, score, high-confidence flag	Records every rule outcome, including rejected pairs.

17.4 Propagation edge, node, and location fields

Edges

Each row represents one directed `source_train_id` to `affected_train_id` relation. It contains the number of causal events and locations, mean gap, maximum source variation, mean/maximum post-worsening, mean incremental worsening, maximum/mean causal score, a sample location, first/last interaction time, and `edge_weight`.

Nodes

The node table contains `out_degree`, `in_degree`, summed outgoing and incoming weights/events, total degree/weight, and the categorical node role. Degrees count distinct counterpart trains; event totals count aggregated high-confidence pair-events.

Locations

The location table stores STANOX/name, pair count, distinct source/affected counts, mean post/incremental worsening, mean gap, maximum causal score, and the location score in Equation (10.9).

17.5 Validation output fields

The control-results table identifies the source and affected train, sample location, reconstructed source/affected interaction times and variations, affected pre/post changes, original edge weight,

number of controls, control post-worsening rate, control mean post-change, and final `control_result`.

Sensitivity and negative-test tables are aggregate summaries. Their recurring statistics mean:

`n_pairs` qualifying interaction pairs under that rule;

`pct_post_worsened` percentage with positive post-change;

`pct_post_stronger_than_pre` percentage whose post-change exceeds pre-change;

`mean/median post change` centre of the post-interaction change distribution;

`unique locations/sources/affected` coverage, not evidential strength;

`top locations` the five most frequent qualifying locations.

17.6 Recovery output fields

Train and location plans retain the aggregates used in ranking, assumed effectiveness, estimated avoided impact, priority score, rank, and `selected_for_intervention`. The combined solution reduces these to `intervention_id`, type, target, benefit, affected count, priority score, intervention cost, and `selected_by_bip`.

Despite the historical column name `selected_by_bip`, the current implementation uses exact dynamic programming rather than an external BIP solver. The summary records solver name, candidate/selection counts, budget, used budget, maximum interventions, estimated total avoided impact, selected type counts, and assumed effectiveness.

17.7 Filesystem output inventory

With `data` selected as the workspace, the principal paths are:

Relative path	Contents
<code>data/railway.db</code>	Persistent SQLite workspace.
<code>data/raw/schedule_full.json.gz</code>	Downloaded full timetable.
<code>data/raw/train_movements_YYYYMMDD_HHMMSS.jsonl</code>	Captured live movement envelopes.
<code>outputs/tables/validation_control_comparison.csv</code>	Full matched-control output.
<code>outputs/tables/validation_threshold_sensitivity.csv</code>	Nine-cell sensitivity output.
<code>outputs/tables/validation_negative_tests.csv</code>	Negative-test output.
<code>outputs/tables/recovery_intervention_plan_by_train.csv</code>	Train ranking.
<code>outputs/tables/recovery_intervention_plan_by_location.csv</code>	Location ranking.
<code>outputs/tables/recovery_intervention_bip_solution.csv</code>	Combined solution.

Relative path	Contents
outputs/tables/recovery_interventi on_bip_summary.csv	Solver summary.
outputs/reports/railconnect-repor t-*/railconnect_report.html	Timestamped HTML report.
outputs/reports/railconnect-repor t-*/tables/*.csv	Complete report-specific snapshots.

Technical Architecture and Runtime Behaviour

18.1 Application layers

The program separates presentation from analysis:

1. PySide6 page classes create controls, tables, status messages, and plots.
2. worker objects run long operations outside the main GUI thread;
3. service modules perform network, SQLite, pandas, graph, validation, optimisation, and reporting work;
4. files and SQLite tables form the boundary between pipeline stages.

This is why the interface remains responsive during large database and analysis operations. Progress signals cross back to the GUI thread, where cards, buttons, logs, and charts are updated safely.

18.2 Threading and close protection

Database builds, train analysis, propagation, validation, recovery, and report generation use background `QThread` workers. Buttons are temporarily disabled to prevent duplicate runs. If the user attempts to close the main window while one of these workers is active, the application blocks the close and asks the user to wait.

Live capture is long-running by design. A normal application shutdown asks the capture worker to stop and close its resources. The safest practice remains to select **Stop capture** explicitly and wait for confirmation first.

18.3 Core dependencies

Dependency	Role
Python	Runtime and standard-library networking, files, SQLite, dates, ZIPs.
PySide6	Native desktop interface, threads, widgets, dialogs, and application lifecycle.
requests	Authenticated streaming timetable download and credential test.

Dependency	Role
stomp.py	STOMP 1.2 live movement subscription.
pandas	Tabular transforms, aggregation, SQLite reads/writes, and CSV exports.
matplotlib	Embedded delay, validation, and recovery charts.
networkx	Propagation graph construction and force-directed layout.
SQLite	Local persistent relational workspace; no separate database server.

The recovery solver does not require SciPy or a commercial optimiser. It is a project-defined exact dynamic programme over quarter-unit integerised costs.

18.4 Network and endpoint behaviour

The connection test and timetable download use HTTPS. The live feed uses STOMP 1.2 at `publicdatafeeds.networkrail.co.uk:61618`, topic `/topic/TRAIN_MVT_ALL_TOC`, automatic acknowledgement, 10-second incoming/outgoing heartbeat settings, and TCP keepalive.

Firewall, proxy, captive-portal, DNS, service-account, or upstream maintenance conditions can affect these connections independently. A successful timetable credential test is not a continuous monitor of STOMP capture health.

18.5 Performance characteristics

The most expensive propagation step enumerates close-time pairs within each location. If many trains occur at the same STANOX and the maximum gap is large, the number of comparisons can rise quickly. Conceptually, a location group of n events has up to $n(n-1)/2$ ordered event pairs before gap, identity, and direction filters. The actual count is lower when the inner loop reaches the maximum time gap early.

Performance and memory therefore depend on:

- capture duration and event count;
- concentration of events at busy STANOX locations;
- maximum interaction gap and direction filtering;
- number of edges admitted to validation;
- candidate limit, budget, and intervention limit in recovery.

For a first test, use a moderate capture and default settings. For a long capture, allow the pipeline to finish without opening other memory-intensive applications. Use the log and Windows Task Manager to distinguish slow progress from a stopped process.

Troubleshooting and Recovery Procedures

19.1 First diagnostic sequence

For any failure, avoid repeatedly clicking the same button. Instead:

1. Read the page status bar and the final lines of its log.
2. Read the terminal traceback, if one appears.
3. Confirm the selected data directory and the expected input file.
4. Confirm that the prerequisite page completed successfully.
5. Preserve the failing source file and a screenshot before changing data.
6. Restart only after any active worker has stopped.

19.2 Startup and environment problems

Symptom	Likely cause	Corrective action
NomodulenamePySide6 or another package	Wrong Python interpreter or inactive virtual environment	From the project root run <code>.venv\Scripts\activate</code> , then reinstall requirements if needed.
Nomodulenamedesktop_app	Command run outside project root	Change directory to the folder containing <code>desktop_app</code> , then run the module command.
PowerShell blocks activation	Local execution policy or using a Command Prompt instruction in PowerShell	Use Command Prompt, or use the correct PowerShell activation script under the user's approved policy.
Application opens and closes immediately	Unhandled startup error	Start from a terminal, keep it open, and inspect the full traceback rather than double-clicking.

Symptom	Likely cause	Corrective action
Text or controls appear unusually large/clipped	Windows display scaling, font substitution, or a window below the intended usable size	Maximise the application, verify normal display scaling, and scroll within result panes where provided.
Old interface appears after code changes	A different project copy/interpreter is running	Confirm the current directory, then use the interpreter-location check below.

From the project root, this command prints the application copy imported by the active interpreter:

```
python -c "import desktop_app; print(desktop_app.__file__)"
```

19.3 Connection and live-feed problems

Symptom	Likely cause	Corrective action
Credentials rejected	Incorrect username/password, account not provisioned, or upstream response	Re-enter manually, use Show briefly to check typing, then retest. Do not paste credentials into logs or screenshots.
Connection test times out	Network, proxy, DNS, firewall, or service outage	Verify general connectivity and that HTTPS access is permitted; retry later without changing local data.
Green indicator but no live messages	Indicator reflects only the last credential test; STOMP may be blocked or quiet	Read the Live Feed log, wait through a known operating period, and verify TCP port 61618 is permitted.
Message count stays zero	Subscription not established, no messages, or authentication failure	Stop, review the error, retest credentials, then start one capture.
Capture file exists but has malformed-line warnings during load	Partial write, manual edit, or unexpected feed content	Ensure capture was stopped cleanly; preserve the original, then use a known complete file.
Cannot close while capture is active	Worker is shutting down cleanly	Select Stop capture and wait for the final stopped message before closing.

19.4 Timetable and database problems

Symptom	Likely cause	Corrective action
Timetable download leaves <code>.part</code>	Download interrupted or failed before atomic replacement	Keep the log/terminal error, verify disk space and network, then rerun the download.
Build says timetable missing	Wrong data directory or download not completed	Select the <code>data</code> directory containing <code>raw/schedule_full.json.gz</code> .
Build reports no/very few schedules	Invalid, truncated, or wrong gzip file	Redownload the full timetable; do not rename an unrelated archive to the expected name.
Movement load reports zero rows	Wrong JSONL file, empty capture, or no valid movement messages	Inspect the file size and capture metrics; capture again if necessary.
Earlier movements disappear after loading another file	Load intentionally replaces <code>train_movements</code>	Reload the desired source file or maintain separate data workspaces for separate captures.
Database cannot be opened/written	File locked, permissions, antivirus, disk space, or read-only directory	Close other SQLite tools, confirm writable local storage and space, then retry.

19.5 Analysis-stage problems

Symptom	Likely cause	Corrective action
No train candidates	No movement table or no usable train IDs	Complete movement load and verify Database counts.
Train ID returns no events	Typo, different capture, or ID omitted from usable rows	Refresh candidates and select an offered ID; comparison is exact.
Unresolved/unknown locations	STANOX not matched to timetable reference data	Verify timetable was built and inspect raw STANOX coverage; some feed codes may genuinely lack a description.
No same-location pairs meet rule	Capture too small, maximum gap too strict, or same-direction filter excludes interactions	Confirm data, then change one documented parameter at a time; do not relax rules merely to force a result.
Propagation completes with no edges	Pairs exist but candidate/post/incremental/score rules reject them	Inspect pair counts and thresholds; report zero as a valid result.

Symptom	Likely cause	Corrective action
Network appears incomplete	Default view intentionally shows a connected high-weight subnetwork of up to 15 edges	Use Strongest edges for the broader ranked table; expanded view enlarges the same curated subnetwork.
Network labels overlap	Dense spring layout or small window	Expand, zoom, and drag; use tables for exact identification.
Validation says run propagation first	Required enriched/edge tables are absent	Run propagation successfully in the same workspace.
Many cases have no control	Control window/filters and available traffic leave no admissible train	Report the no-control count; a wider window changes the method and must be documented.
Recovery says no edges	Propagation table absent or contains no high-confidence edges	Resolve/interpret propagation first; recovery cannot operate independently.
Selected count is below maximum	Budget binds, candidates have large costs, or no positive-benefit candidates remain	Check used budget and costs; maximum is a cap, not a target.

19.6 Chart, table, report, and export problems

Symptom	Likely cause	Corrective action
Chart is cut off vertically	Window/result viewport smaller than chart content	Maximise the window and use the result-pane scrollbar. Charts are placed in scrollable result areas.
Table columns extend beyond view	Wide analytical schema	Use the horizontal scrollbar; in HTML, scroll inside each table container.
Load existing results fails	Relevant tables have not been created in this workspace	Run that stage once or select the correct data directory.
Report inventory shows missing sections	Those analysis tables do not exist; partial reports are allowed	Run the missing stages if required, refresh inventory, then generate a new report.
Open latest report does nothing after restart	Latest-report pointer is session-local	Generate a report in this session or open the existing HTML directly from <code>outputs/reports</code> .
Export button unavailable/-fails	No current report, Downloads unavailable, or filesystem error	Generate first, verify Downloads exists/is writable, then retry.
ZIP name has <code>-copy-2</code>	A file with the original name already exists	Expected collision protection; no file was overwritten.

Symptom	Likely cause	Corrective action
HTML preview shows only 20 rows	Deliberate preview limit	Open the associated CSV in the report's <code>tables</code> folder for all rows.

19.7 When to stop rather than retry

Stop and preserve evidence if a traceback mentions database corruption, a source file unexpectedly shrinks, results change without an input/parameter change, or the application writes outside the selected project/output paths. Make a copy of `railway.db`, the relevant source file, terminal output, and settings before further experiments.

Security, Privacy, and Safe Operation

20.1 Credential model

The application does not write the Open Data username or password to disk. They remain in the page's input widgets for the running process, are passed to short-lived network workers as needed, and worker-held strings are cleared when the operation finishes. This reduces persistence risk but does not make the process memory a secure secret vault.

Use these practices:

- use only the official account intended for this data service;
- never commit credentials, paste them into source code, or put them in a screenshot, issue, report, CSV, or terminal command history;
- use Hide after checking a password and close the application on a shared machine;
- do not distribute a live capture or analysis report before reviewing what operational identifiers and locations it contains;
- protect exported ZIPs using the organisation's normal data-handling controls.

20.2 Local storage and backups

SQLite, raw source files, reports, and CSVs are unencrypted by the application. They inherit the protection of the Windows account, filesystem, backups, and device. Store the project in a user-controlled local folder, avoid public cloud-sync folders unless intentionally approved, and keep a backup before large reruns if the current results must be preserved.

20.3 Network trust boundary

Only credentials intended for Network Rail Open Data should be entered. A locally modified copy of the source code can change endpoints or handling, so review changes before running untrusted forks. TLS protects the HTTPS request; the STOMP connection is governed by the configured public feed endpoint and local network controls.

20.4 Data-source terms and attribution

Network Rail’s Open Data page states that feed access requires an account and is governed by terms that may change. It also states that the feeds belong to Network Rail Infrastructure Limited, that applications using them must not be described as “official,” and that availability or total uptime is not guaranteed. Review the current terms before acquiring, publishing, or redistributing data. Do not use Network Rail or partner branding in a way that implies endorsement.

This project is an independent prototype and is not affiliated with or endorsed by Network Rail. See the source links in [Appendix B](#); those pages, rather than this manual, are authoritative for current access and reuse conditions.

20.5 Operational safety

RailConnect 2000 is not connected to signalling, traffic management, driver advisory, or control-room command systems. It must not be used to issue movement authorities or make real-time safety decisions. Any operational interpretation requires independent railway expertise, current official information, and a validated decision process outside this application.

Statistical Interpretation and Model Limitations

21.1 Descriptive quantities are not causal estimates

Timetable variation, pre/post changes, pair counts, graph weights, and control rates are descriptive features of the captured data. The application uses temporal ordering and same-location proximity to construct hypotheses. It does not estimate a causal effect using randomisation, an identified natural experiment, a structural timetable model, or a fully adjusted observational design.

In particular, the causal score S is a weighted rule, not:

- a probability that one train delayed another;
- a calibrated classifier probability;
- a p-value, likelihood ratio, or confidence level;
- the number of delay minutes transferred;
- evidence of responsibility or fault.

21.2 Potential confounding and dependence

Source and affected trains may both respond to a third cause: signalling or infrastructure disruption, adverse weather, station congestion, possession, crew/rolling-stock circulation, timetable conflicts, or a third train. Repeated events from the same train, route, location, and disruption are not independent observations. Consequently, a large event count must not be treated as a large independent sample without a dependence-aware design.

21.3 Selection and measurement limitations

Results are conditional on:

- which messages arrived during the chosen capture window;
- successful parsing and location enrichment;
- the timetable snapshot and STANOX mapping;
- missing or repeated feed events;
- chosen thresholds and fixed internal constants;

- use of adjacent observed events as local before/after context.

The method does not impute missing movements. A next observed event may not be the next operational event if messages are missing. Variation values are accepted from the feed rather than recomputed from planned and actual times.

21.4 Raw variation and status limitation

TRUST supplies a numeric `timetable_variation` value and a separate `variation_status` such as `EARLY`, `LATE`, or `ONTIME`. The current application stores both fields, but its analytical stages use the numeric value directly and do not convert it to a signed value from the status. Accordingly, a positive analysis value must not automatically be read as “minutes late.” This affects train trajectories, threshold tests, post-event changes, causal scores, validation outcomes, and recovery rankings.

The status column should therefore be inspected alongside the numeric value. A future implementation should normalise the pair into one documented signed measure—for example, late positive, early negative, and on-time zero—and then revalidate every dependent rule. That normalisation is a recommendation, not a feature of the audited build.

21.5 Timetable-integration limitation

The CIF download is parsed into schedule and schedule-location tables, while the TIPLOC file is used to enrich movement locations. In the audited build, the downstream analytical services do not match a TRUST train identifier to a specific CIF schedule or use planned paths, calling patterns, operating dates, day-of-week rules, or STP indicators in propagation inference. References in this manual to timetable context therefore describe stored reference data, not a structural timetable model.

21.6 Matched-control limitations

Controls are selected by location, direction, time window, and an exclusion rule. There is no multivariate propensity model, route equivalence check, service-class match, exact timetable path matching, uncertainty interval, or hypothesis test. The four validation classifications are deterministic comparisons, not statistical significance decisions.

Validation also reconstructs each case from the aggregate source/affected train identifiers and a sampled location name. It chooses the smallest positive gap found under those constraints rather than preserving the exact event pair and STANOX that originally produced an aggregated propagation edge. It does not reapply every propagation threshold during that reconstruction. Validation is therefore a diagnostic of a reconstructed case, not an independent confirmation of the exact original event pair.

The fixed five-minute upstream exclusion threshold and fixed sensitivity/ negative-test grids are part of the current implementation. Changing the UI control window does not change those fixed constants.

21.7 Sensitivity and negative tests

Sensitivity results show how descriptive counts/outcomes vary under a small, predefined threshold grid. They do not prove robustness beyond that grid. Negative tests are comparative diagnostics: a surprising result can expose common-cause structure, overly permissive rules, or data artefacts, but a favourable contrast still does not establish causation.

21.8 Graph limitations

Graph arrows inherit the pair rule and score. The network is directed but not necessarily acyclic: different events can support opposing or cyclic relations. The displayed subnetwork is selected for readability and high weight, not as an unbiased sample. Spring-layout distance and orientation carry no railway, geographical, or temporal meaning.

21.9 Optimisation limitations

The recovery model optimises the score-derived estimated benefit under a dimensionless cost rule. It assumes a common effectiveness factor and additive benefits. It does not model:

- overlapping benefit when selecting both a train and a location;
- mutually exclusive or prerequisite interventions;
- staffing, rolling stock, infrastructure, or timetable feasibility;
- uncertainty in propagation edges or effectiveness;
- network feedback after an intervention;
- monetary cost, passenger disbenefit, safety, or equity constraints.

An optimal solution is optimal only for Equation (12.6), its candidate set, and its assumptions.

21.10 Responsible wording for results

Prefer:

“Under the documented close-time, same-location heuristic, train A was an inferred source for train B in k high-confidence events. Matched-control and negative-test results provided [classification/summary].”

Avoid:

“Train A caused train B’s delay,” “the probability of causation was S ,” or “the recovery plan will save b minutes.”

Replace k , S , and b with values only while retaining their documented units and caveats. If rigorous inferential claims are required, the next research stage should pre-specify an estimand, identify confounders, model dependence, quantify uncertainty, and validate against independent operational records.

Default Settings and Quick Reference

22.1 Default controls

Page	Control	Default	Meaning
Propagation	Maximum gap	10 min	Largest admitted source-to-affected interval.
Propagation	Source delay	5.00 min	Minimum source variation for candidate status.
Propagation	Affected delay	5.00 min	Minimum affected interaction variation.
Propagation	Causal score	8.00	Minimum rule score for high confidence.
Propagation	Same direction	Checked	Requires equal direction indicators.
Validation	Maximum cases	50	Highest-weight propagation edges to attempt.
Validation	Maximum interaction gap	10 min	Largest reconstructed positive source/affected gap.
Validation	Control time window	30 min	Symmetric time window around the affected event.
Recovery	Total budget	8.00 units	Maximum sum of heuristic intervention costs.
Recovery	Maximum interventions	5	Cardinality cap.
Recovery	Assumed effectiveness	0.50	Multiplier applied to edge-weight totals.
Recovery	Candidates per type	100	Highest-ranked trains and locations admitted before filtering.

22.2 Fixed internal constants

The following are not currently exposed as controls:

- propagation short-gap bonus uses $[10 - \Delta t]_+$, even when the maximum gap control is not 10;
- validation excludes a potential control if another train within the preceding maximum interaction gap has variation at least 5 minutes;
- sensitivity grid uses gaps $\{5, 10, 15\}$ minutes and delay thresholds $\{3, 5, 10\}$ minutes, with same-direction matching;
- negative tests use a 10-minute gap and 5-minute delay threshold;
- large-gap negative test uses $10 < \Delta t \leq 30$ minutes and does not apply a direction filter;
- network display is capped at 15 connected high-weight edges;
- analytical tables display at most the first/top 200 rows where stated;
- HTML reports preview 20 rows per table while CSVs contain all rows;
- recovery costs are integerised in quarter units for dynamic programming;
- the raw numeric variation is used without signing it from `variation_status`;
- validation reconstructs a case from train IDs and a sample location instead of retaining the exact propagation-event identity.

22.3 Startup command card

```
cd "C:\path\to\Network-Rail-Delay-Propagation-Pipeline"  
.venv\Scripts\activate  
python -m desktop_app.main
```

To finish, stop live capture if active, wait for all analysis buttons to become enabled, close the window, and then close the terminal. Deactivation of the virtual environment is optional after the process has ended:

```
deactivate
```

Frequently Asked Questions

Does “Open Data Connected” mean the live feed is currently healthy?

No. It means the last explicit credential test was accepted. Use Live Feed status, log messages, and increasing message counts to assess capture.

Can I close the terminal after starting the application?

No. In the current source-mode workflow the Python process belongs to that terminal. Keep it open until the GUI has closed. A future packaged executable can remove this requirement.

Are credentials saved?

No credentials are deliberately written to disk. They remain in process memory/input widgets for the session, so still handle the running machine securely.

Can I analyse multiple movement captures at once?

The current movement loader replaces its table with the selected file. To combine captures, create a documented preprocessing workflow or separate workspaces; do not assume repeated Load operations append.

Why are there fewer network edges on screen than in the metric?

The explorer deliberately selects a connected high-weight subnetwork of up to 15 edges. The edge table and SQLite results contain the full inferred network.

Does Expand network show the whole raw network?

No. It opens a larger, interactive view of the same curated subnetwork and adds the legend.

Why can validation evaluate fewer cases than the maximum?

Maximum cases is a cap. An edge may not be reconstructable under the current gap/direction rules, and only available high-weight edges can be attempted.

Does a positive timetable-variation number always mean the train is late?

Not in the current analysis. The feed also supplies `variation_status`, but the audited analytical code does not use that field to sign the numeric value. Inspect both columns and do not interpret a positive plotted value automatically as lateness.

Does the full CIF timetable drive propagation matching?

No. The application stores schedule and schedule-location tables, but the current downstream analysis uses timetable-derived TIPLOC data only for STANOX-to-name enrichment. It does not yet match TRUST trains to full CIF schedules or planned paths.

Why can selected recovery actions be fewer than the requested maximum?

The budget or positive-benefit candidate set can bind before the cardinality limit. The optimiser is not required to fill all slots.

Where are the complete report rows?

In the timestamped report directory's `tables` folder and in the exported Downloads ZIP. The HTML displays only the first 20 rows per table.

Can an old report be opened after restarting?

Yes, by opening its `railconnect_report.html` directly in File Explorer. The GUI's latest-report pointer itself is only maintained for the current run.

Can I cite a causal score as probability?

No. It is an uncalibrated heuristic score whose terms and coefficients are defined in Equation (10.6).

Can the recovery plan be used operationally?

Not without independent validation and a real intervention/constraint model. It is a prototype prioritisation experiment.

Glossary

Term	Definition in this project
Actual time	UTC timestamp reported for an observed movement event.
Affected train	Later train in a same-location pair being examined for subsequent worsening.
Candidate propagation	Pair whose source and affected variations meet their minimum thresholds.
Causal score	Heuristic weighted score in Equation (10.6); not a probability.
Control event	Movement event from another train near the affected event in time/location/direction that passes the exclusion rule; counts are not necessarily distinct trains.
CRS	Three-letter Computer Reservation System station code where available in timetable data.
Edge	Directed aggregation from one inferred source train to one affected train.
Edge weight	Heuristic aggregation score in Equation (10.8).
Enrichment	Joining movement STANOX values to timetable-derived display names.
High-confidence pair	Candidate satisfying positive post-change, positive incremental change, and score threshold.
Incremental worsening	Post-interaction change minus pre-interaction change.
JSONL	One JSON value per line; capture rows are receipt-time/message envelopes.
Location score	Heuristic importance score in Equation (10.9).
Matched control	Admissible comparison train selected near the affected interaction.
Node	Train participating in one or more inferred directed edges.
Post-interaction change	Affected next-event variation minus interaction-event variation.
Pre-interaction change	Affected interaction-event variation minus previous-event variation.
Propagation	Project hypothesis that delay may transmit between close-time same-location trains.
Recovery benefit	Assumed-effectiveness multiplier times aggregated edge weight; dimensionless.
Recovery cost	$1 + 0.25$ times affected-train count; dimensionless quarter units.

Term	Definition in this project
STANOX	Numeric railway location identifier used for movement/timetable matching.
STOMP	Messaging protocol used to subscribe to the live movement topic.
Source train	Earlier train in a same-location temporally ordered pair.
Spring layout	Force-directed graph placement used for readability, not geography.
TIPLOC	Timing Point Location identifier from timetable data.
Timetable variation	Raw numeric number of minutes supplied by the feed. Its separate status/direction is stored in <code>variation_status</code> ; the audited analysis does not combine them into a signed delay.
TRUST	Train running/movement information source represented by the live feed.
Workspace	Selected <code>data</code> directory and its associated database/raw/output paths.

Sources and Version Basis

This revision was checked against the local RailConnect 2000 source snapshot available on 25 September 2026. The repository did not provide a tagged application release or pinned dependency lock file, so behaviour can change in later commits or under later library versions. Record the Git commit hash and `pip freeze` output for any result that must be reproduced.

Project source

<https://github.com/faust2/Network-Rail-Delay-Propagation-Pipeline>

Empirical case-study source

Supplied report archive, inspected 27 September 2026; authoritative export `railconnect-report-20260902-070325`, generated 2 September 2026 from analysis records dated 6 June 2026. The archive includes the HTML report and its CSV result tables but not the originating movement JSONL, capture log, or SQLite database.

Network Rail Open Data feeds and current access conditions

<https://www.networkrail.co.uk/who-we-are/transparency-and-ethics/transparency/open-data-feeds/>

Network Rail transparency and data-feeds licence information

<https://www.networkrail.co.uk/who-we-are/transparency-and-ethics/transparency/>

Open Rail Data Wiki: Train Movement field reference

https://wiki.openraildata.com/index.php/Train_Movement

Web sources above were checked on 25 September 2026. Their owners may update them. The wiki is a technical community reference; the Network Rail pages and applicable terms govern feed access and reuse.

Formula Summary

With $[x]_+ = \max(x, 0)$:

$$\begin{aligned}
W_{\text{pre}} &= V_0 - V_-, \\
W_{\text{post}} &= V_+ - V_0, \\
W_{\text{inc}} &= W_{\text{post}} - W_{\text{pre}}, \\
T &= \min([V_s]_+, [W_{\text{post}}]_+), \\
S &= 0.6[V_s]_+ + 1.8[W_{\text{post}}]_+ + 1.2[W_{\text{inc}}]_+ + 0.4[10 - \Delta t]_+ \\
&\quad + 0.8T + \mathbf{1}\{W_{\text{inc}} > 0\} - 0.6[W_{\text{pre}}]_+, \\
E_{uv} &= N_{\text{events}} + 0.7N_{\text{locations}} + \overline{W}_{\text{post}} + 0.8\overline{W}_{\text{inc}} + 0.2V_{s,\text{max}} - 0.2\overline{\Delta t}, \\
L &= N_{\text{pairs}} + 0.5N_{\text{affected}} + \overline{W}_{\text{post}} + 0.8\overline{W}_{\text{inc}} - 0.2\overline{\Delta t}, \\
b_u &= e \sum_v E_{uv}, \quad b_l = e \sum_{(u,v) \text{ at } l} E_{uv}, \\
c_i &= 1 + 0.25a_i, \\
\max_x \quad &\sum_i b_i x_i \quad \text{s.t.} \quad \sum_i c_i x_i \leq B, \quad \sum_i x_i \leq M, \quad x_i \in \{0, 1\}.
\end{aligned}$$

Every term above is defined in the corresponding functional chapter. The formulae describe the implementation; they do not add statistical calibration or operational validity beyond the limitations already stated.